```
    pq.insert((dist, node.point))                          #11
    if compare(target, node) < 0
then                                  #12
      closeBranch ← node.left
      farBranch ← node.right
    else
#13
      closeBranch ← node.right
      farBranch ← node.left
    pq ← nNearestNeighbor(closeBranch, target, pq)
#14
    (nnnDist, _) ← pq.peek()
#15
    if splitDistance(target, node) < nnnDist
then                      #16
      pq ← nNearestNeighbor(farBranch, target,
pq)                     #17
    return
pq                                                          #18
```

#1 Find the n points in the k-d tree that are closest to a given target.

#2 Initialize a max-heap (or any other (max) priority queue), bounded in its size, so that it will only contain the n smallest elements added to it. Refer to chapter 2, section 2.7.3, to see how insertion in such a queue works.

#3 Before starting our search, we need to initialize the priority queue by adding a “guard”: a tuple containing `infinity` as distance, a value that will be larger than any other distance computed, and so it will be the first tuple removed from the queue, if we find at least n points in the tree.

#4 We start the search on the root, using a recursive internal function.